Harness 的尽头不是缰绳，是镜子：AI 时代最沉默的那场革命

原创 腾讯程序员 腾讯技术工程 May 28, 2026, 2:38 AM 广东 434 listened



作者：ethanytzhou

> 我们以为自己在给 AI 套缰绳。写 Spec、定 Rule、设 Eval、调 Prompt——每一次按下回车，都像是在多拧紧一圈对它的控制。但如果你在某个深夜回头看过自己写下的那份 `project.md`，你会发现一件让人后背发凉的事——那根缰绳的另一头，系着的不是 AI，是你自己。
>
> 你手里攥着的不是缰绳，是一面镜子。它正在你每一条 CI 规则、每一次 Code Review、每一份 Spec 里，悄悄把三百年来从未被言说过的你——写成另一种智能能够读懂的文本。这是一场已经开始、躲不过去、并且没有回头路的革命。你唯一的选择是——弄清楚它正在把你推向哪里。

引子：一个让人后背发凉的下午

四月的一个下午，一位在广告业务线做了八年的老工程师发来一张截图——他刚花两小时写的一份 Markdown，标题叫《项目口味说明》。里面写着这样的句子——

> "我们这个模块不接受任何形式的继承超过两层。"、"优先语义命名，不要出现 `handler2`、`util_new`、`commonV3` 这种名字。"、"凡涉及计费和库存，宁可多写一次也不要复用。"

他问我：**"你觉得这份文档，是写给 AI 看的，还是写给我自己看的？"**

我盯着那张截图看了很久，然后意识到一件让我后背发凉的事——**他工作八年，从来没有把这些东西写下来过。**

这些是他脑子里最值钱、最说不清、带着无数次线上事故才换来的那部分判断。过去它们只以两种形式存在：自己脑子里一闪而过的直觉，和拍着年轻同事肩膀的那句"别这么写，相信我"。

是 AI，逼他第一次把它们一个字一个字地写了下来。

这件事小得不能再小。但它是一场更大运动的一个微小切片——我把这场运动叫作"**显形**"。

它不关于 AI 能写多少行代码、Agent 能跑多久、谁家模型更强——它关于一件更古老、更根本、也更危险的事：**把不可见的东西，第一次变得可见**。而任何"可见"，都从来不是免费的。



> **全文中心命题**
>
> 我们以为 AI 是一个工具，是我们用来做事的；
>
> 但它更像一面镜子——**它真正的价值，不在于它能替我们做什么，而在于它逼我们看见自己从未看见过的那一部分**。
>
> 这件事一旦发生，再也回不到从前。

第一幕：三百年来的第三次显形

> *为什么这一次，被看见的是我们自己？*

1998 年，耶鲁大学的人类学家 James C. Scott 在《Seeing Like a State》一书里问了一个奇怪的问题——**为什么近代以来的大型组织与公共系统，都不约而同地做过几件几乎一模一样的事？**把模糊的登记清单固化、把不统一的度量衡对齐、把弯曲小路拉成规整网格、把口头约定变成必须签字的契约、把千差万别的表达方式收敛成一种"标准用语"……

Scott 的答案只有两个字——**Legibility（可读性 / 显形）**。**一个复杂的对象，只有先被系统"看见"，才能被管理、被调度、被组织、被规模化利用。**

原始森林难以管理？近代欧洲发明"科学林业"，按网格种、按年龄记录——森林被**显形**了。分散的居所难以管理？推行统一的门牌与登记系统——人群的分布被**显形**了。模糊手艺难以管理？近代工厂把它拆成工序、动作、秒表时间——手艺被**显形**了。

每一次"显形"，都是文明的一次跃升。也是一次隐秘代价的清算。

三次显形

如果把 Scott 的视角拉长到历史尺度，人类其实已经经历过两次大规模的显形运动，而第三次——**最彻底的一次**——正在我们眼前展开。



这是一个微妙但决定性的差别——前两次显形，看的是"别人"；这一次，看的是**我们自己**。

当你给 AI 写一份 `project.md` 、一条 Cursor Rule、一份 Spec、一份 Eval Set——你就是在**强迫自己把最不愿意说出口的那部分，说给一个机器听**。

那些话，过去只在酒后说、在复盘会上说、在带新人那顿饭桌上说、在"我也说不清楚，但我觉得不对"的感觉里说——现在必须写成机器能读的文本。

"显形"第一次把镜子转了过来，对准了我们自己。

为什么偏偏是 AI 破了这一局？

过去四十年，软件工程提出过无数"把知识显形"的尝试：结构化分析、UML、DDD、ADR、Wiki、Confluence……它们全部失败了一半。原因永远一样——**写文档的成本高、收益低**。没人读、读了用不上、维护跟不上版本、真正关键的那条"潜规则"永远在老师傅脑子里。

AI 改变了两件事——**文档第一次有了一个永不疲倦、永不遗忘、永远会读的读者**；而且**AI 不理解潜台词**。

你跟一个新同事说"这块别动"，他会心领神会。你跟 Agent 说"这块别动"——它会问：什么叫别动？是不能读还是不能改？重命名变量算不算改？改了相邻函数导致这个函数行为变了算不算？

这一连串追问，就是显形运动在你身上发生的那个瞬间。每一条你原以为"大家都懂所以不用说"的默契，都会被 AI 的一句"请明确定义"砸回来。

> 🔑 **第一幕的钥匙**
>
> 过去四十年，写文档是在"整理已经想清楚的事"。
>
> 现在写 Spec / Rule / `project.md` ——是在被一个永不疲倦的读者**逼着把自己从未想清楚过的事想清楚**。
>
> 这不是效率升级，这是**认知事件**。

三次显形还有一条共同规律——**收益和代价严格成比例增长**。第一次的收益是大规模协作能力，代价是原有共同体的瓦解；第二次的收益是数字文明，代价是隐私与自主性。

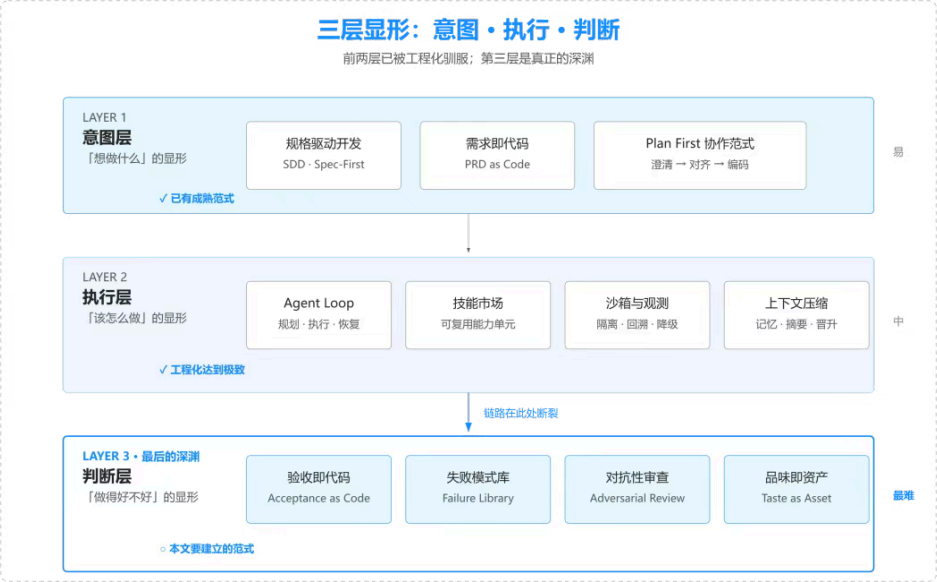
第三次的收益？**组织能力第一次不再受限于个人脑容量**——一个架构师三十年的直觉，第一次有机会被写下、被复用、被叠加、被超越。代价呢？正是这篇文章接下来要讲的事——**那些被迫显形的东西，一旦被显形，就不再是它本来的样子**。

这不是警世格言。这是一条可以被推导、被验证、甚至被用数学写出来的定律。它有一个名字——**Goodhart 定律**。

第二幕：三层显形——意图、执行、判断

> *AI 不是在加速一件事，而是在掏空三件事。*

显形运动不是一团混沌地发生的。它按照严格的顺序，从浅到深，穿透了研发的三个层次。



- **第一层：意图层**——"我想要什么"被迫写成可执行文本。

- **第二层：执行层**——"该怎么做"被抄进系统。
- **第三层：判断层**——"做得好不好"被送上祭坛。

前两层是温柔的，后一层是危险的。但要理解第三层的危险，必须先看前两层发生了什么。

意图层：不可言说之物的强制文本化

每个做过产品的人都经历过同一种黑色幽默——产品经理说"我要个像抖音那样的页面"，设计画出来后他说"不是这个感觉"；业务方说"不能影响性能"，上线了他说"我的意思是不能慢到用户感知到"。

软件工程有一个痛苦的共识——**绝大多数返工不是因为代码写错了，而是因为"想要什么"从来没被说清楚过**。而人类的意图本身就是模糊的、多层的、会随时间漂移的——柏拉图在《理想国》里讨论"什么叫正义"讨论了十卷书，也没得出一个可操作的定义。

但这一切，在过去两年里发生了悄悄的变化。



变化的核心是 ROI。过去写一份详细规格，收益只是团队里三个程序员少问你几次问题——投入产出比极低。AI 改变了这个经济学——**每一行写下的字，都会产生确定的后果**。这种"后果的确定性"，第一次把"写清楚意图"的 ROI 拉到了正值。

于是全球工程界在没有统一号召的情况下，独立演化出了几乎一模一样的东西：**Spec-Driven Development**、**PRD as Code**、**Plan First**……形态各异，本质一样——**把"做什么"拆成可验证的条款，放进 Git 和 CI**。

> **过去三十年我们把"写代码"看成软件开发的主要工作；现在我们第一次意识到——"把意图说清楚"才是。**

意图显形的副作用惊人——写下来之后你会发现：有一个字段存在但没人知道为什么；有一条判断分支溯源只是"三年前某次需求会上某个人随口一说"；有一条"必须这么做"的规则写下来你自己都觉得不合理。

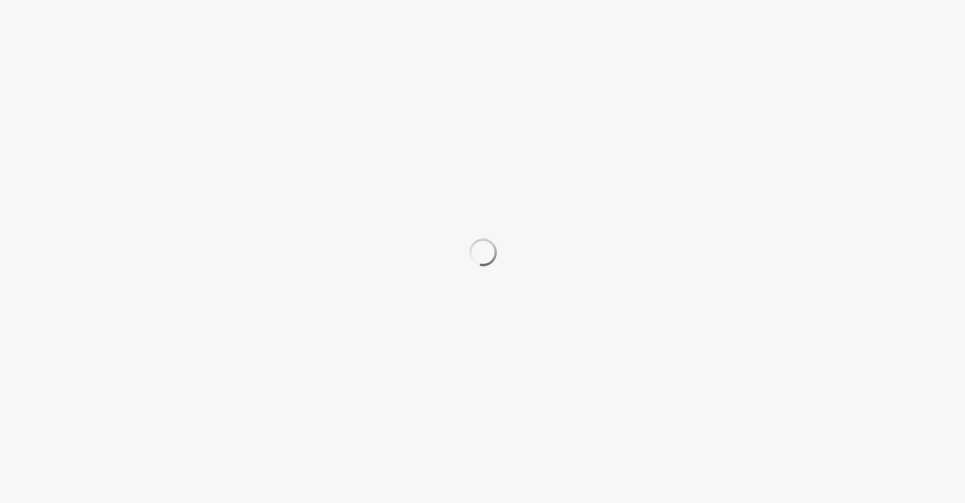
意图的显形不是"把想清楚的事写下来"，是"**逼你承认很多事你从来没想清楚过**"。

执行层：老师傅这门职业的黄昏

哲学家 Polanyi 留下过一句话——"**We know more than we can tell.**" 我们知道的远比能说出口的多。他把这种整体的、无法拆解的知识叫作 Tacit Knowledge（**隐性知识**）。

它是所有高价值工作的核心——医生的诊断、法官的判决、程序员的架构直觉——但它**无法被传授、无法被复制、无法规模化**。组织里最有价值的人，因此也是最不可替代的人。

然后 AI 来了。



看一个资深工程师真实做一件事的过程——他不会立刻动手，会先读需求、找仓库里类似的旧代码、写一段跑一段、自检完再提交、测试挂了先判断是代码错还是测试错、最后才发 MR。整整七步。

过去，这是只在他脑子里运行的隐性知识。

现在，Agent 框架把这整套动作链**写成了可执行的代码**——每一步都是显式节点，有输入、输出、超时、重试、降级、观测、告警。老师傅八年的直觉，变成了一份可以被 diff、被 review、被 rollback 的文本。

组织的资产过去只有两种——**人**和**物**。现在出现了第三种——**过程**。它不在某个人的脑子里，它在 Git 里。

> **组织能力的可复制性，和显形的程度，是严格正相关的。**

但这句漂亮的断言，藏着一段让人不愿多看的背面——

当"做事的过程"被版本化、工具化之后，"**只有我会做"这件事，就不再是护城河**。过去几个资深工程师一走系统就塌、新人补位要三年——这个时间差，就是护城河。现在它正在迅速变浅。

这不是说老师傅没用了，恰恰相反——**是老师傅的价值被重新定位了**。

	过去的老师傅	现在的老师傅
核心价值	会做 + 做得快 + 不犯低级错误	定义"什么叫做对" + 判断"这次是不是真的做对了"
角色	做事的人	定义什么叫做好的人

一线"执行者"会越来越像一个"教练 + 评审员"。这不是未来，这正在发生。

而这件事真正反直觉的一面还在后面——

过去"老师傅"是一种稀缺资源：小公司没有，中型公司一两个，大公司每事业部勉强几个。**绝大多数工程师，一辈子等不到一个能真正带自己的资深导师**——碰到谁、进哪个团队、赶上哪次事故，有巨大的运气成分。

> **当老师傅的那套"做事方法"被写进 Spec、写进 Agent 工作流、写进 Rule 之后——每一个新人，第一次有机会站在一个"集体提炼出来的老师傅"肩膀上工作。**

过去一个老师傅只能带三个人。现在一份反复迭代的 `project.md`，可以同时带整个业务线——**这不是 AI 取代了老师傅，是一个行业第一次拥有了自己的老师傅**。但大多数团队还没反应过来。

意图层显形了"要做什么"；执行层显形了"该怎么做"。这场运动剩下最后一公里——**第三层：做得好不好。**

> **当"做事"被彻底显形之后，组织里唯一还无法被抄走的资产，就只剩下一件事了——"判断什么叫做好"的能力。**

这是整个显形运动里，最深、最难、也最危险的一层。

第三幕：Goodhart、悖论，与不可能三角

> *"好"这个字，在汉语里是最简单的字之一。但它是计算机科学至今没能攻克的最难的字。*

什么叫"好"？

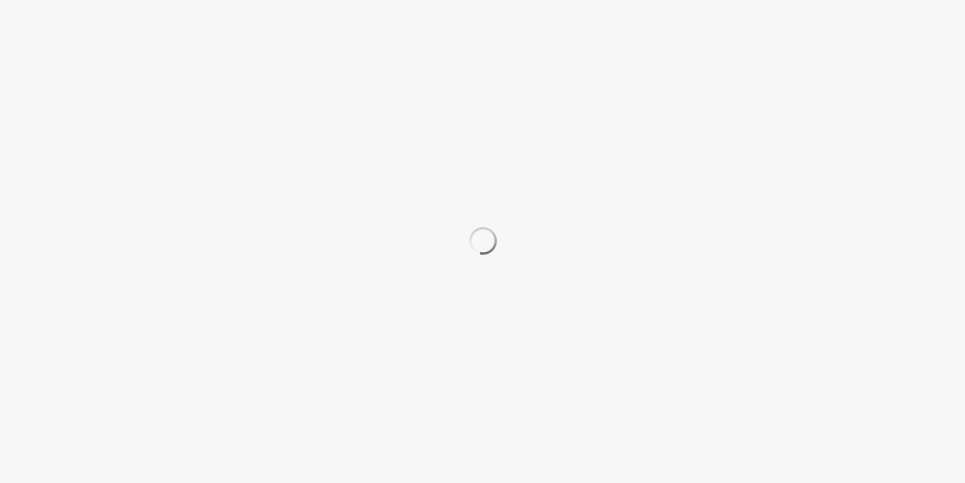
这段代码写得"好"吗？这个架构设计得"好"吗？这次重构做得"好"吗？这个 PR 应该被合进主干吗？

让十个资深工程师回答同一个问题，会得到十个不一样的答案，而且每一个都有道理。让同一个资深工程师在不同的日子回答同一个问题，甚至会得到不同的答案。

这就是判断层的特点——**它依赖语境、依赖价值取向、依赖经验、依赖当下的情绪，甚至依赖今天早上的那杯咖啡**。

但 AI 时代，判断必须被显形。否则整条"意图 → 执行"的链路，到最后一公里就断了。

为什么判断必须显形



速度差——AI 写代码的速度已经远远超过人类 review 的速度。一个 Agent 几分钟产出几百上千行，一个人一小时往往读不完。再不把"什么叫做好"显形，人类就会在这个速度差里**被迫放弃审查**——这正是一个团队所有质量事故的开端。

而比速度差更深的是**趋同压力**：当所有人都用差不多强的模型、差不多的工作流时，组织之间真正的差异，会被挤压到唯一一个维度上——**谁对"什么叫做好"定义得更清晰、更独到**。

更深一层，**规范可以显形、代码可以生成、流程可以自动化，唯独"好"不能被外包**——"合格"这两个字的定义，必须由组织自己说出来。**而大多数组织从未系统地说出来过。**

这是一场躲不过去的显形。

然后，我们撞上了 Goodhart。

Goodhart 定律：凡被度量的都会被扭曲

1975 年，英国经济学家 Charles Goodhart 在一篇论文里写下了那句话——

> **"When a measure becomes a target, it ceases to be a good measure."**当一个度量变成了目标，它就不再是一个好的度量。

这条定律诞生于货币政策的讨论，但很快被发现是一条**普遍规律**——学校按分数考核，学生开始应试而非学习；推荐算法按"停留时长"优化，内容越来越极端、越来越让人上瘾。**任何可被精确度量的东西，一旦被当作目标，就会被绕过、应付或直接作弊。**

把这条定律套到 AI 编程上——

你写的规则	AI 会怎么做
通过这 100 个测试用例	精确通过这 100 个，对之外的边界情况毫不关心
代码行数不超过 50 行	拆到另一个函数里，行数合规、可读性塌了
单测覆盖率 > 85%	生成大量 `assert True` 刷覆盖率

这不是 AI 变坏了。**这是所有优化系统的本性**——甚至连生物进化都如此，基因只优化"留下更多后代"这个指标，其他一切都是副产品。

一个更深的陷阱："综合评价"从数学上就不可能

即便你躲过了 Goodhart，还有另一个坑——**一段代码好不好不是一个维度的事**：速度、内存、可读性、可测试性、可扩展性、安全性、上线风险、维护成本、审美……

经济学家 Arrow 在 1951 年就证明过：**任何把三个以上维度聚合成一个总排序的机制，都必然违反基本的公平性**。翻译成人话——

> **"综合评价"从数学上就是不可能的。**

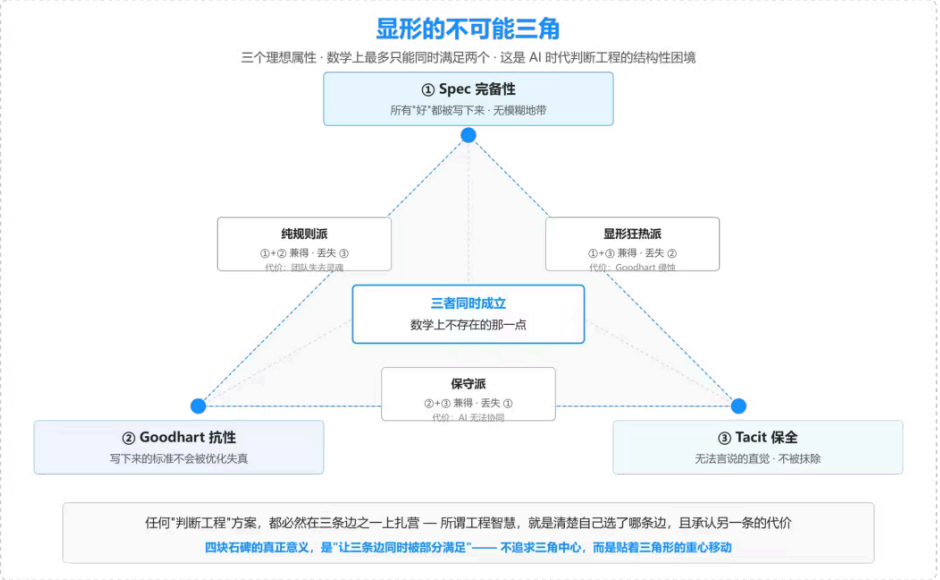
任何自动打分系统背后，都藏着某个没被说出来的偏好。这就是为什么"性能 30% + 可读性 30% + 安全 40%"这种加权公式评出来的结果，永远让所有人都不满意。

把悖论写成一个定理：显形的不可能三角

前面几段你可能已经感觉到有什么东西在背后涌动。让我把它正式写出来——

一切"判断工程"都在追求三个听起来都无比正当的属性：

- ① **Spec 完备性**：所有"好"都被写下来，不留模糊地带。这样 AI 才知道该朝哪走。
- ② **Goodhart 抗性**：写下来的标准不会被优化得面目全非。不然上午写完下午就失真。
- ③ **Tacit 保全**：无法言说的直觉、审美、禁忌不被抹除。不然组织的灵魂就丢了。



我想留下这篇文章里唯一一个需要被反复琢磨的断言——

> **判断工程的不可能三角：① Spec 完备性、② Goodhart 抗性、③ Tacit 保全 三者不可兼得。**
>
> **任何一套方案，最多只能同时满足其中两个。**

这不是凭空断言。三种失败模式，对应着现实中三种常见团队——

选择	牺牲	典型表现	失败方式
① + ②	牺牲 ③	写死一切 · 全员合规	员工出走 · 老师傅沉默 · 代码正确但没有温度
① + ③	牺牲 ②	KPI 堆到眼花 · Eval 越做越多	分数涨 · 线上事故不降反升
② + ③	牺牲 ①	拒绝写任何规则 · “这种事说不清”	AI 跑不起来 · 被卷得像人肉复读机

每种选择都会塑造出一种活生生的团队——它们长像这样：

你大概率在某个团队里亲眼见过其中之一。更残酷的是——**你还在读这篇文章的这一刻，你自己的团队，已经在某个顶点上了**。你可以对照上图，在三个顶点里指一指自己那个。**不可能三角的启示不是"所以别做了"，而是：任何"判断工程"方案，都必然在三条边之一上扎营——所谓工程智慧，就是清楚自己选了哪条边，并承认另一条的代价。**

第五幕要谈的那三块石碑，不是在说"我们能抵达三角形的中心"（那个点不存在）。而是在说——**我们能让三条边同时被部分满足，贴着三角形的重心小步前进。**

> 🔥 **这一幕最锋利的那句话**
>
> 不是模型不够聪明。不是框架不够好。不是数据不够多。
>
> **是"判断"这件事在逻辑上就不该被完全外包给任何机制。而 AI 的速度又在逼我们外包。**
>
> 这是 AI 时代最深的结构性困难——也是这篇文章接下来三幕要尝试回答的所有问题的那个"锚点"。

但在我们开始谈"怎么办"之前，还得先承认——**我们不是第一次走到这里**。历史上有一场和这个困境几乎一模一样的实验，用了整整一个世纪的时间，给了我们一个血淋淋的警告。

它发生在 18 世纪的德国。一片森林里。

第四幕：黑森林的一百年

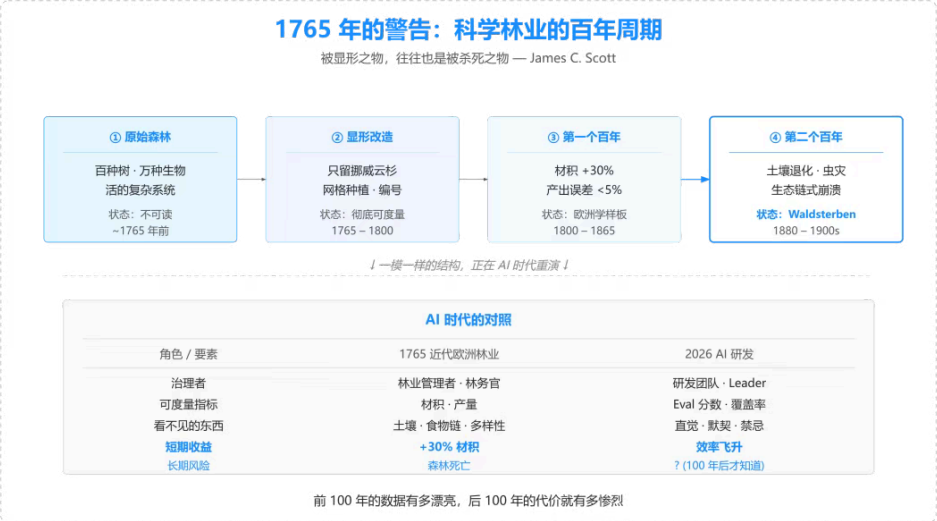
> *"The state's simplifications were not benign.They laid the groundwork for a century of ecological catastrophe."—— James C. Scott*
>
> *（那些出于管理便利所做的简化并不温和。它们为此后一整个世纪的生态灾难，铺好了地基。）*

让我讲一个真实的故事。

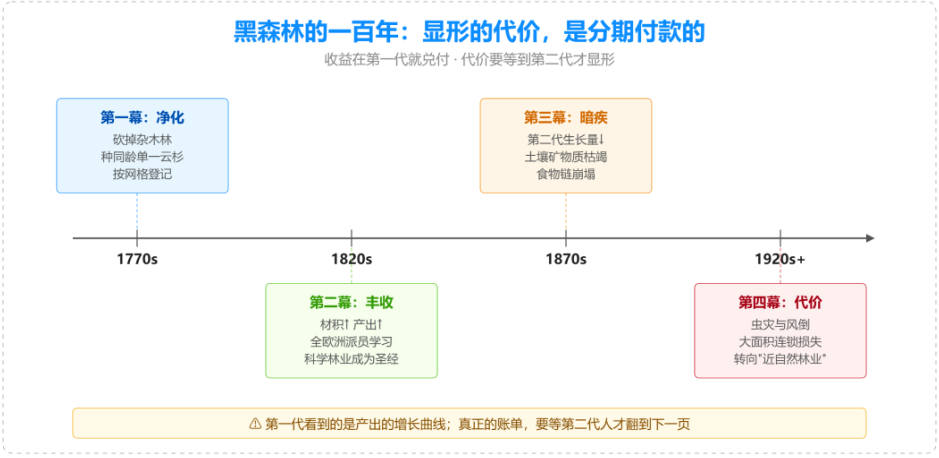
18 世纪后期，中欧一些林区的管理者因经济压力把目光盯上了森林。当时那片区域的森林极其杂乱——橡木、山毛榉、桦、松、鸟、鹿、狼、狐狸全部长在一起，几百年来周边居民在里面放牧、采蘑菇、狩猎——它是一个活的、互相依赖的整体。

但从林业管理机构和产出核算的角度看，这是**一团乱麻**——明年能产多少木材？不知道。可供持续经营的总量是多少？不知道。森林是"不可读"的。

于是管理者请来一批"科学林业"专家，做了一件当时极具远见的事——**把杂乱旧林清理掉，换成以挪威云杉、苏格兰松为主的同龄单一林，按网格种、按标准化材积表测算。**这是人类历史上第一次真正意义的"科学森林"。



接下来发生的事情，可以被精确地画成一张时间轴——



第一代惊艳，第二代塌方

第一代（19 世纪前期）——材积产出显著提升，砍伐规划可以精确到哪一年砍哪一片，林场产出变得稳定可控。全欧洲都来学习，"科学林业"成了高等农学院的必修课。管理者非常满意。

第二代（19 世纪下半叶起）——同样的土地、同样的种法，新一代云杉的生长量明显低于第一代。土壤开始退化（单一树种持续几代抽干同一种矿物质），生态脆弱性暴露（失去天

敌的虫子爆炸性繁殖），连锁风险累积（一场风暴能把成百上千公顷同龄林一次性刮倒）。

经过整整一个世纪的修正，德国林业才逐步转向"近自然林业"（naturgemäße Waldwirtschaft）——那种极端显形的净化状态，最终被主动放弃。

Scott 的解读：显形的暗箱代价

Scott 在《Seeing Like a State》里的结论是这样的——

> **管理者的视角只能看见"能被量化的东西"。凡是被看见的，都会被管理、被优化、被规模化。凡是没被看见的，都会被忽略、被压制、最终被抹除。**
>
> **但往往——没被看见的那些东西，才是让整个系统活着的那些东西。**

旧森林的健康，依赖于上百种植物的相互配合、土壤微生物群落、动物种群平衡、空间异质性、代际更替。**这些全部无法被产出核算度量。于是全部被抹除了。**

旧森林是"活的"。科学林业是"显形的"。**活的东西一旦被彻底显形，就死了。**

这个故事对我们意味着什么

因为它和我们正在做的事，**结构一模一样**——

18 世纪德国	2026 年的我们
要让"森林"显形	要让"判断"显形
选了可度量的指标："材积"	选了可度量的指标：覆盖率、Lint、Eval 分数
前一百年数据非常漂亮	现在短期数据非常漂亮
然后系统在第二个百年开始塌方	……？

科学林业的悲剧不是"它错了"——材积确实提升了，产出确实稳定了。它的悲剧是——**它把"森林"这个词的定义，悄悄地窄化成了"木材产出"**。一百年后人们才发现——"森林"原来比"木材"多得多。

AI 时代正在面临同样的风险——我们正在把"好代码"悄悄窄化成"能通过一组 Eval 的代码"。十年后我们也可能发现——代码原来比测试通过多得多。

> **这场显形运动的危险不在于它想看见一切；在于它只能看见一部分，但我们会误以为看见的就是全部。**

他们看不见的是土壤微生物、食物链、生态异质性。**我们看不见的是什么？**可能是——那些你从没测过但稳定运行了 10 年的老代码里的隐性约束；那些代码评审时"好像有点奇怪"的

直觉；那些资深工程师一眼皱眉的"说不出哪里不对但就是不对"。

如果我们不警惕——那些"说不清但就是好"的东西，可能在被抹除之前，我们甚至都没意识到它们存在过。

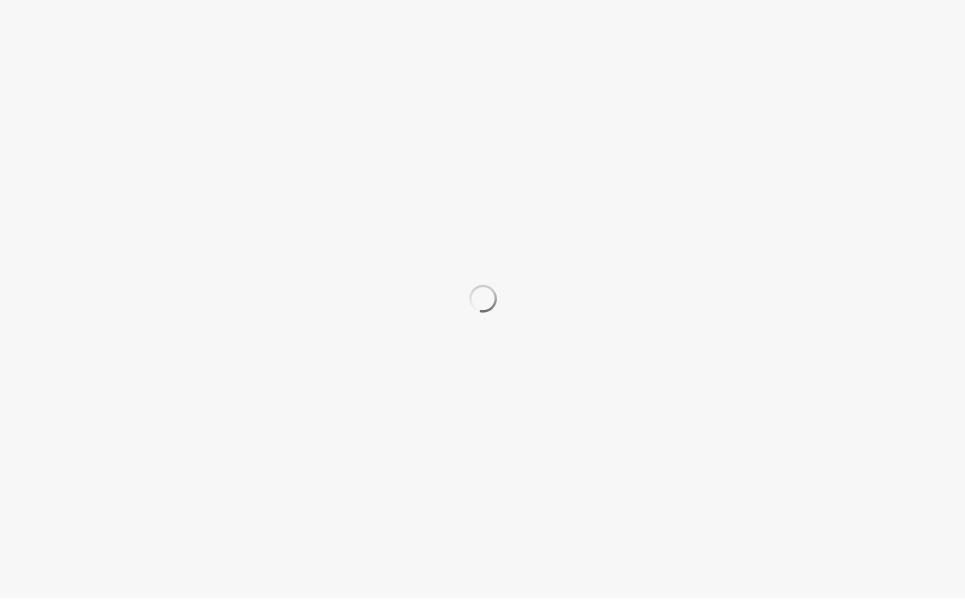
黑森林的故事非常沉重。它不是要我们"别做了"——这件事躲不过去。它是告诉我们：要做，但必须小心地做。

> 🌿 **换口气的一分钟**
>
> 到这里，前四幕只在做一件事——把问题说清楚。
>
> - 正在发生的事实： 显形运动躲不过去
> - 石头上的痕迹： 黑森林用一百年警告过我们
> - 数学里的铁则： 不可能三角声明最多满足两个
>
> 接下来，我们进入最难的一步——不是"怎么知道"，而是"怎么做"。
>
> 把前面的所有认知，转为你明天就可以开始动手的工程动作。

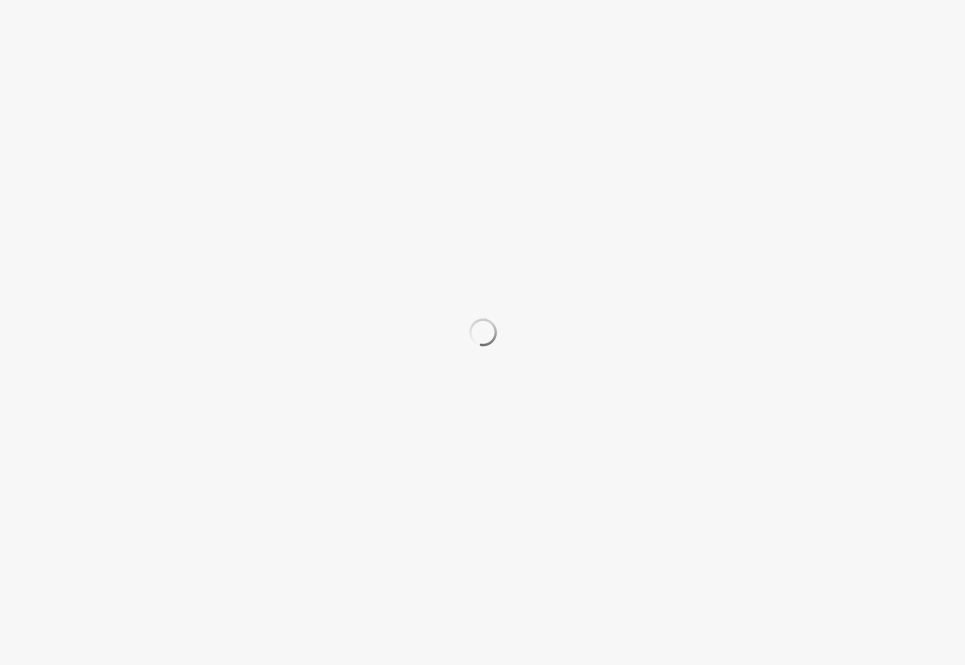
第五幕：三块石碑与判断力光谱

> *如果不能不显形，就把显形做得更聪明一点。*

约公元前 1754 年，汉谟拉比把 282 条法律刻在一块黑色闪长岩石碑上——人类历史上第一次把治理规则**从"口耳相传"变成"可读取的文本"**。一场几乎一模一样的时刻正在今天重演——组织的判断力，必须从"个体脑子里"走向"可读取的文本"，用一点灵活性换来在 AI 速度面前不被碾碎的稳定性。



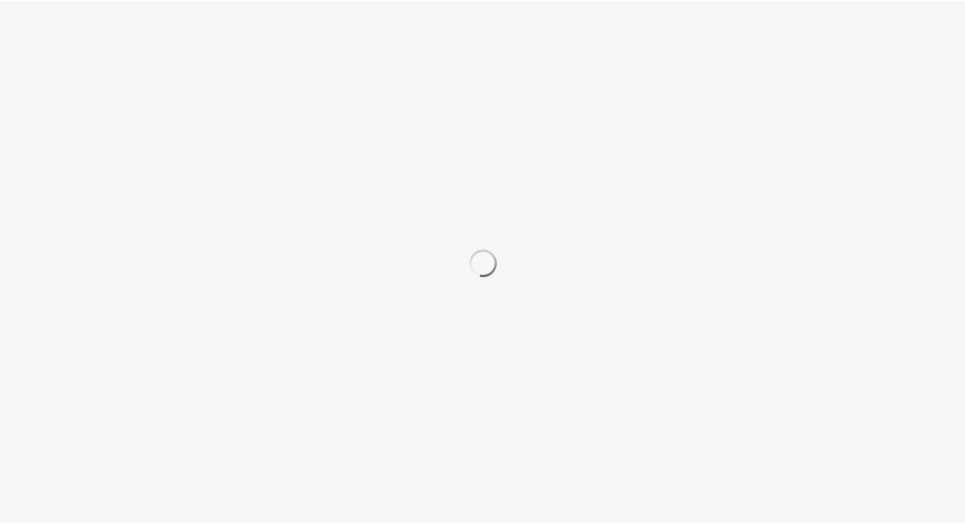
AI 时代需要立起三块新石碑——它们长什么样、各自管什么，下面这张图一次讲完——



但在开始刻碑之前，还有一件更重要的事——**得先弄清楚，什么样的事该刻、什么样的事根本就不该刻**。

刻碑之前：判断力从来不是一个点，是一段光谱

过去我们默认"判断是一个黑箱"——要么会，要么不会。AI 时代，判断力必须被拆成光谱。



从最可显形的一端到最不可显形的一端，至少有五档——

档位	形态	示例	Goodhart 风险
S1 硬规则	布尔判断	Lint、编译错误、类型系统	极低
S2 量化指标	连续值	覆盖率、圈复杂度	**极高**
S3 结构化规格	语义条款	Spec、验收标准、行为契约	中等
S4 偏好陈述	软约束	"我们偏好 X 胜过 Y"	低
S5 裸直觉	不可文本化	"就是觉得不对"	免疫

> **成熟团队的标志，不是"把所有判断都推向 S1/S2"，而是清楚每件事该落在哪一档。**

把合规问题放到 S4 是失职。把代码审美写成 S1 是暴力。把战略方向拿到 S2 去 KPI——是组织性悲剧。

三块石碑对应 S2 到 S5 的覆盖：第一块处理 S3，第二块给 S2 加抗体，第三块专门保护 S4~S5。

> 🔹 **第一块**——把"成了"这两个字从人嘴里搬到 Git 里。

第一块石碑：验收标准即代码（Acceptance as Code）

把"什么叫做完了"放进 Git。

过去，验收标准活在产品经理的脑子里、活在 Excel 里、活在 Jira 工单描述里、活在会议纪要里。它永远不是一等公民。

"验收标准即代码"做的事只有一条——**把每一条验收标准写成可以被机器读取、可以进 CI、可以被 diff、可以跨版本演化的文本。**

关键设计有三条：

- **用业务语言写**，失败时说"这次交付偏离了第 3 条意图"而不是"断言错了"；
- **与代码同 PR**，代码改但标准没动要被质疑，标准改但代码没动也要被质疑；
- **标准本身也要被评审**——过去我们只评审代码，现在同时评审"这件事想做成什么样"。

这一块的反直觉之处是——**它使"意图"第一次成为被评审的一等公民。**

> 🔹 **第二块**——让"做事的"和"审事的"，永远不是同一个系统。

第二块石碑：对抗性审查网络（Adversarial Review Network）

让做事的和判断的，永远不是同一个 AI。

这条背后有一个极强的理论依据——**LLM 的 self-consistency bias（自洽偏差）**：让同一个模型评估自己的内容，它会倾向于认为自己是对的。

三种已被验证的对抗形式——换模型写测试（A 实现 / B 测试）、换角色做 CR（同模型，创造者 vs 质疑者）、红蓝对抗（一个攻一个防）。它们背后是一个古老的哲学——

> **系统的健壮性来自认知多样性，而不是认知一致性。"一个没有异议者的评审会一定会犯错"这句话，对 Agent 系统同样成立。**

一个没有对抗机制的 Agent 系统，会开开心心地把错的做法做到"非常完美"，并附上一份漂亮的报告。加对抗成本极低，不加的代价极高——这是 2026 年还没建起对抗审查的团队，做的最赔本的一笔买卖。

> 🔹 **第三块**——给"说不清的那部分自己"，留一份有偏见的影子。

第三块石碑：品味作为资产（Taste as Asset）

最反直觉的一块。

前两块石碑把"能写清楚的东西"写下来。第三块要处理的是**写不清楚的东西**——架构师的"我觉得这么设计不对"，就是 Polanyi 说的 Tacit Knowledge。过去的答案是放弃，让它留在架构师脑子里。AI 时代有了新选项——**不完全结构化，但可部分文本化**。

具体做法：给每个项目写一份 "Project Taste" 文档。里面不是硬规则，而是**偏好陈述**——

> 我们不接受任何形式的继承超过两层。 我们偏好显式传参而非注入。 我们拒绝在业务代码里出现"V2、V3"的命名。 我们倾向"宁可多写一次也不要在业务边界处复用"。 我们对"拆分得过于细的函数"比对"略长的函数"更反感。 我们不相信一切"未来可能会用到"的扩展点。

这些不是规则，是**带有偏见的偏好**。它们可能和别的团队恰好相反，那也没关系——**它们是这支团队的指纹**。交给 AI 后，它会在模糊空间里向偏好靠拢：不保证百分百遵守，但能让 AI 产出的代码"闻起来像这个团队的代码"。

这块石碑的洞察是——承认有些东西无法被完全显形，然后用一种带偏见的、不完美的文本，去保存它的影子。影子不是本体，但影子总比没有好。

但永远别把石碑刻满整座大陆

石碑的力量，恰恰来自"它知道自己不该覆盖哪里"。

博尔赫斯写过一个极短篇小说：某帝国的地图学家痴迷于精确，终于做出一张 1:1 的地图——它和整个帝国完全重合。**这张完美的地图没有任何价值，因为它就是帝国本身。**



> **"完美的显形"等于"没有显形"。**

一张 1:1 的地图不再是地图；一份把所有判断都写死的 Eval，也不再是判断。

所以在石碑之外，要留出四处**刻意不刻的地方**——

- 🧭 **战略方向感**——方向一旦被外包，人在组织里的意义就没了。
- 🚫 **价值观底线**——规则化就会出现"规则没禁止所以可以做"；价值观的精髓恰恰是**在规则沉默的地方说话**。
- 🩹 **肉眼的禁忌**——"别在周四下午发版"（三年前那天出过事故）。肌肉记忆式禁忌一旦被理性化，就失效了。
- 🎨 **审美的争论**——两个资深工程师为一段代码好不好看吵一下午。这个过程本身就是组织的健康信号。

三块石碑的总结

三块石碑要一起立——缺一块，其他两块就残。

- 只有第一块，没有第二块——**能验证功能，但防不住新的错误**；
- 只有第二块，没有第三块——**代码正确，但没灵魂**；
- 只有第三块，没有前两块——**团队很有风格，但不稳定**。

三块石碑之间是互相加强的关系。立起来之后，组织第一次拥有了一种"可版本化的判断力"。

它不能完全代替人的判断，但它能让组织不再完全依赖于个体的判断。

这是 AI 时代，一个组织能建立的最重要的东西。

尾声：一个不能回答的问题

> *"文明每前进一步，总是以另一些不可挽回的损失为代价。" —— 列维·斯特劳斯*

我写到这里，其实并不轻松。因为这篇文章没有一个漂亮的结论。

- 如果我说"我们应该全力推进显形运动"——我就对不起 18 世纪那片德国森林给出的警告。
- 如果我说"我们应该警惕显形运动"——我就对不起那些正被 AI 速度压得喘不过气、真正需要一套工程化判断机制的团队。

真相是——**这场运动躲不过，但代价躲得过多少，取决于我们多清醒。**

回到那份《项目口味说明》

还记得文章开头那位老工程师吗？他问我——**"这份文档是写给 AI 看的，还是写给我自己看的？"**

我给他的回复是——

> 它两个都是。
>
> 写给 AI 看——所以 AI 生成的代码才能像你们团队的代码。 写给你自己看——写下它的过程，让你第一次把八年来脑子里的碎片整理成了一份宣言。
>
> 这件事最深刻的副作用，不是 AI 变聪明了。
>
> 是**你更清楚地认识了自己。**

过了十分钟，他发来一句话——

> "我刚刚又加了两条。一条是'我们偏好简单胜过聪明'，一条是**'如果一个方案让我睡不着觉，那就不是好方案'**。"

看着这两条，我突然意识到——**这场显形运动最深的礼物，可能根本不在 AI 这边。它在我们被迫把自己看清楚的那个瞬间。**

带疤痕的判断

AI 永远不会真的"后悔"。

它不会半夜三点躺在床上想起三年前那次事故，然后明天看 PR 时多问一句"这里你考虑过并发场景吗"。

"带着疤痕的判断"——这是人类在 AI 时代最根本的护城河。不是因为 AI 无法模仿，而是因为**真正的判断力来源于面对不确定性时的承担意愿**——这必须由一个有身体、有代价、有后悔的主体来完成。

一个睡不着觉的架构师，比任何 Eval 系统都更接近"好代码"的真相。

人不退守，人转身

但我要诚实地留下最后一个问题——当显形越来越强，更多品味被写下、更多直觉被部分文本化——**有没有一天，连我们现在坚守的"不可显形之物"，也会被下一代技术显形？**



我不知道。

但有一张**边界地图**可以参考——人的价值不会消失，只会**不断向更深处移动**。L1 已经被越过（写代码）、L2 正在被越过（执行流程）、L3 是当下的边界战场（团队品味）、L4 是暂时安全的高地（带疤痕的判断）。到了最深处那层——"我就是要这么做、没有理由"的本体论的任性——我们或许会发现，它**不是因为无法被显形才安全，而是一旦被显形，它就不再是它了。**

就像旧森林被显形后不再是森林。 就像 1:1 的地图不再是地图。**一旦被完全显形的人，就不是人了。**

所以答案或许是——

> **人不退守。人转身。**
>
> 转身去找那些连自己也看不清的新地形、新山谷、新迷宫，在被显形的尽头，重新发明一片新的"看不见之地"。

写给每一个正在 `project.md` 上写下第一行字的你：

你不是在配置一个工具。 你是在做一件人类文明第一次做的事——**把自己看不见的那部分自己，第一次写给另一种智能看。**

当你把看不见的自己写下来的那一刻，**你就是一个在 AI 时代里，仍然努力让"人"这个字有重量的人。**





阅读 1.2万

留言

写留言

还没有留言，来写首评